

Programación Orientada a Objetos

Ejercicio 1.

Crea la clase Fracción. Los atributos serán numerador y denominador. Y algunos de los métodos pueden ser invierte, simplifica, multiplica, divide, etc.

Ejercicio 2.

Crea la clase Tiempo con los métodos suma y resta. Los objetos de la clase Tiempo son intervalos de tiempo y se crean de la forma `Tiempo t = new Tiempo(1, 20, 30)` donde los parámetros que se le pasan al constructor son las horas, los minutos y los segundos respectivamente. Crea el método `toString()` para ver los intervalos de tiempo de la forma 10h 35m 5s.

Si se suman por ejemplo 30m 40s y 35m 20s el resultado debería ser 1h 6m 0s. Realiza un programa de prueba para comprobar que la clase funciona bien.

Ejercicio 3.

Defina una clase que sirva para representar el estado de una bombilla (encendido o apagado). Defina, asimismo, dos métodos que permitan encender (on) y apagar (off) la luz de la bombilla.

Para probarlo, cree un método `main` que cree un objeto de la clase definida, que haga uso de los métodos previamente definidos y vaya mostrando el estado de la bombilla.

Ejercicio 4.

Defina una clase 'Monedero' que permita gestionar la cantidad de dinero de que una persona dispone en un momento dado. La clase deberá tener un constructor que permitirá crear un monedero con una cantidad de dinero inicial y deberá definir un método para meter dinero en el monedero, otro para sacarlo y finalmente, otro para consultar el disponible; solo podrá conocerse la cantidad de dinero del monedero a través de este último método. Por supuesto, no se podrá sacar más dinero del que haya en un momento dado en el monedero.

Para probar el funcionamiento de la clase, escriba un método 'main' con una serie de instrucciones que hagan uso de los métodos definidos.

Ejercicio 5.

Se quiere definir una clase que permita controlar un sintonizador digital de emisoras FM; concretamente, lo que se desea es dotar al controlador de una interfaz que permita subir (up) o bajar (down) la frecuencia (en saltos de 0.5 MHz) y mostrar la frecuencia sintonizada en un momento dado (display).

Supondremos que el rango de frecuencias a manejar oscila entre los 80 Mhz y los 108 MHz y que al inicio, el controlador sintoniza a 80 MHz. Si durante una operación de subida o bajada se sobrepasa uno de los dos límites, la frecuencia sintonizada debe pasar a ser la del extremo contrario.

Ejercicio 6 (Propiedad estática).

Queremos modelar una casa con muchas bombillas, de forma que cada bombilla se puede encender o apagar individualmente. Para ello haremos una clase Bombilla con una variable privada que almacene si está encendida o apagada, así como un método que nos diga si una bombilla concreta está encendida.

Además queremos poner un interruptor general de la luz, tal que si saltan los fusibles, todas las bombillas quedan apagadas. Cuando el fusible se repara, las bombillas vuelven a estar encendidas o apagadas, según estuvieran antes del percance.

Ejercicio 7 (Propiedad estática)

Para probar el funcionamiento de la clase, escriba un método 'main' con una serie de instrucciones que hagan uso de los métodos definidos.

Se desea representar el conjunto de bombillas que hay en una casa, así como la potencia consumida en un momento dado por todas ellas. Para ello, defina una clase 'Bombilla' que tenga al menos dos atributos:

Uno para identificarla y otro que indique la potencia que consume; también, deberá crear dos métodos que permitan encender y apagar la bombilla. La clase deberá disponer de un mecanismo que permita obtener la potencia consumida por el conjunto de bombillas que estén encendidas en un determinado instante.

Ejercicio 8 (Propiedad estática)

Crea una clase Votacion que permita llevar el control de unas elecciones entre distintos candidatos. De cada candidato se necesita saber el nombre y el número de votos que lleva.

Realiza la clase con los atributos necesarios y los métodos que permitan conocer la cantidad de votos de cada candidato, así como el nombre del candidato más votado y cuantos votos ha conseguido. Crear la clase PruebaVotacion donde se creen varios candidatos, se simule unas pequeñas elecciones y se muestre el resultado.

Ejercicio 9 (Propiedad estática y tipo enum)

Crea la clase Pizza con los atributos y métodos necesarios. Sobre cada pizza se necesita saber el tamaño - mediana o familiar - el tipo - margarita, cuatro quesos o funghi - y su estado - pedida o servida.

La clase debe almacenar información sobre el número total de pizzas que se han pedido y que se han servido. Siempre que se crea una pizza nueva, su estado es “pedida”.

El siguiente código del programa principal debe dar la salida que se muestra:

```
public class PedidosPizza {  
    public static void main(String[] args) {  
        Pizza p1 = new Pizza("margarita", "mediana");  
        Pizza p2 = new Pizza("funghi", "familiar");  
        p2.sirve();  
        Pizza p3 = new Pizza("cuatro quesos", "mediana");  
        System.out.println(p1);  
        System.out.println(p2);  
        System.out.println(p3);  
        p2.sirve();  
        System.out.println("pedidas: " + Pizza.getTotalPedidas());  
        System.out.println("servidas: " + Pizza.getTotalServidas());  
    }  
}
```

```
pizza margarita mediana, pedida  
pizza funghi familiar, servida  
pizza cuatro quesos mediana, pedida  
esa pizza ya se ha servido  
pedidas: 3  
servidas: 1
```

Ejercicio 10 (Relación entre objetos)

Se desea llevar un control del estado de una cuenta corriente; la cuenta corriente está caracterizada por su saldo y sobre ella se pueden realizar tres tipos de operaciones:

- saldo: devuelve el saldo de la cuenta (puede ser negativo).
- imposición (cantidad): ingresa en la cuenta una cantidad de dinero.
- reintegro (cantidad): saca de la cuenta una determinada cantidad de dinero.
- Traspaso (cantidad, CuentaCorriente): traspasa una cantidad de dinero a otra cuenta recibida por parámetro.

Suponga que la cuenta inicialmente tiene un saldo de cero. Escriba una clase CuentaCorriente que implemente la funcionalidad descrita; escriba un pequeño programa principal para probar su funcionamiento.

Ejercicio 11 (Relación entre objetos)

Defina una clase punto que tendrá dos atributos, de tipo real, x e y, que representarán las coordenadas del punto dentro del plano. Defina un método que tenga como argumento otro objeto de la clase punto y que calcule la distancia entre los dos puntos.

Para probar esta funcionalidad, escriba un método main en el que se creen dos puntos, el primero de los cuales deberá tener las coordenadas 100.0, 200.0 y el segundo deberá tener las coordenadas 400.0, 800.0 y calcule e imprima la distancia entre ambos:

$$\text{distancia}(p1,p2) = \text{raizcuadrada}(\text{cuadrado}(x2-x1)+\text{cuadrado}(y2-y1))$$

Ejercicio 12 (Relación entre objetos)

Crea una clase 'AlarmaDeTemperatura' cuyos objetos activen un objeto 'Timbre' cuando la temperatura llegue a un cierto valor. Para probar la alarma simule en el Main un 'Sensor' que utilice la clase 'AlarmaDeTemperatura' y simule variaciones de temperatura y cambios en la temperatura de alarma. La clase 'Timbre' mostrará un mensaje por pantalla cuando se active y otro cuando se desactive.

Ejercicio 13 (Relación entre objetos)

Crea una clase cubo que contenga información sobre la capacidad y su contenido actual en litros. Se podrá consultar tanto la capacidad como el contenido en cualquier momento.

Dota a la clase de la capacidad de verter el contenido de un cubo en otro (hay que tener en cuenta si el contenido del cubo origen cabe en el cubo destino, si no cabe, se verterá solo el contenido que quepa). Hacer una clase principal para probar el funcionamiento con un par de cubos.

Ejercicio 14 (Array de objetos)

Crear una clase PruebaVotacion2 que permita crear un array de 5 candidatos de la clase Votacion creada en el [ejercicio 8](#). Usar un bucle para generar los objetos pidiendo por teclado el nombre del candidato.

Utiliza otro bucle para generar de manera aleatoria 100 votos y finalmente recorrer el array para mostrar los votos de todos los candidatos, así como el candidato ganador.

Ejercicio 15 (Array de objetos)

Defina una clase que permita almacenar datos sobre los planetas que forman parte del sistema solar. Algunos atributos que puede ser interesante recoger son: la masa del cuerpo, su

diámetro, el período de rotación sobre el propio eje, período de traslación alrededor del sol, distancia media a recorrida en su órbita, ...

Defina un método, que de un determinado planeta del sistema solar, imprima toda la información de sobre el mismo.

A continuación crea una clase PruebaPlaneta2 que permita crear un array de 5 objetos de la clase Planeta. Usa un bucle para generar los objetos pidiendo por teclado el nombre del planeta, y asignar de manera aleatoria valores para la órbita (kilómetros) y traslación (días). Finalmente recorrer el array para mostrar los datos de todos los planetas.

Ejercicio 16 (Array de objetos + Propiedad estática)

Se desea representar los coches que hay en un concesionario, de cada coche se almacenara matricula, modelo, motor, precio y color. Se desea saber en todo momento la cantidad de dinero invertida en el concesionario que será la suma del valor de todos los coches.

Se desea disponer de un método para insertar un nuevo coche aportando todos los datos del mismo, y otro para obtener los datos de un coche (recibe un entero correspondiente a la posición en el array estático y devuelve un objeto coche). También se tendrá un método para listar por pantalla todos los coches del concesionario mostrando todos sus datos.

Realiza una clase con Main para probar todos los métodos de la clase Coche.

Ejercicio 17 (Array de objetos + Propiedad estática).

Modifica el programa “Colección de discos” como se indica a continuación:

a) Mejora la opción “Nuevo disco” de tal forma que cuando se llenen todas las posiciones del array, el programa muestre un mensaje de error. No se permitirá introducir los datos de ningún disco hasta que no se borre alguno de la lista.

b) Mejora la opción “Borrar” de tal forma que se verifique que el código introducido por el usuario existe.

c) Modifica el programa de tal forma que el código del disco sea único, es decir que no se pueda repetir.

d) Crea un submenú dentro dentro de “Listado” de tal forma que exista un listado completo, un listado por autor (todos los discos que ha publicado un determinado autor), un listado por género (todos los discos de un género determinado) y un listado de discos cuya duración esté en un rango determinado por el usuario.

Ejercicio 18 (Array de objetos + Propiedad estática).

Crea el programa GESTISIMAL (GESTIÓN SIMplificada de Almacén) para llevar el control de los artículos de un almacén. De cada artículo se debe saber el código, la descripción, el precio de compra, el precio de venta y el stock (número de unidades). El menú del programa debe tener, al menos, las siguientes opciones:

1. Listado
2. Alta
3. Baja
4. Modificación
5. Entrada de mercancía
6. Salida de mercancía
7. Salir

La entrada y salida de mercancía supone respectivamente el incremento y decremento de stock de un determinado artículo. Hay que controlar que no se pueda sacar más mercancía de la que hay en el almacén.